

ISG 算法插件开发规范

文档版本：v2.0

适用范围：ISG 平台算法插件研发、测试、交付与运维

发布日期：2026-06-15

文档状态：正式版

修订记录

版本	日期	修订说明
v1.0	2026-06-08	初始版本，定义插件结构、参数声明、运行入口和返回值。
v2.0	2026-06-15	重构为工程化规范，补充接口契约、质量门禁、错误码、测试、交付和安全要求。

1. 目的

本规范用于统一 ISG 算法插件的开发方式、接口约定、参数声明、输入输出、异常处理和交付标准，确保插件在平台内具备一致的可发现性、可配置性、可执行性、可观测性和可维护性。

插件开发应遵循以下原则：

- 接口稳定：所有插件必须实现统一入口 `run(payload, context)`，并声明模块级 `PARAMETERS`。
- 配置显式：所有可由平台 UI 配置的参数必须在 `PARAMETERS` 中声明。
- 行为可控：长耗时任务必须支持进度上报、日志记录和取消检查。
- 输出可追溯：所有产物必须写入平台指定目录，并在返回结果中声明来源、路径、状态和元数据。
- 失败可诊断：失败返回必须包含稳定错误码和可读错误信息。
- 交付可验证：插件提交前必须通过静态检查、单元测试、样例运行和交付物完整性检查。

2. 适用范围

本规范适用于以下类型插件：

- 数据清洗与质量检查插件，例如模糊检测、异常标签识别、重复样本识别。
- 数据增强与样本生成插件，例如几何变换、颜色扰动、目标裁剪、合成样本生成。
- 标注辅助插件，例如候选标签推荐、质量评分、标注修复建议。
- 模型前后处理插件，例如特征提取、格式转换、结果过滤。

本规范不直接约束平台核心服务、调度器、UI 前端或独立微服务；但当这些组件调用算法插件时，应遵循本文定义的插件契约。

3. 术语定义

术语	定义
插件	一个符合 ISG 规范的 Python 模块，提供参数声明和统一运行入口。
算法 Key	插件的全局唯一标识，采用点分命名空间，例如 generation.image.geometric_transform。
PARAMETERS	模块级参数声明列表，用于平台生成 UI 配置项和执行前校验。
payload	平台传入插件的执行请求对象，包含算法标识、参数、输入数据、输出目录和任务目标。
context	平台注入的运行上下文对象，提供日志、进度上报和取消检查能力。
output_dir	平台为本次任务创建的输出目录，插件只能在该目录或其子目录写入任务产物。

4. 插件工程模型

ISG 算法插件本质上是一个标准 Python 模块。平台通过动态导入加载插件，并读取模块级变量和入口函数。

每个插件必须满足以下最低要求：

- 文件编码为 UTF-8。
- 模块顶层声明 PARAMETERS: list[dict[str, Any]]。
- 模块顶层定义 run(payload: dict[str, Any], context: Any) -> dict[str, Any]。
- 不得在导入阶段执行耗时计算、读取大文件、访问网络或创建输出文件。
- 不得在 run() 内修改 PARAMETERS。
- 不得依赖当前工作目录；所有文件路径必须来自 payload 或配置项。

推荐目录结构如下：

```
plugins/  
  user/  
    image_geometric_transform.py  
    label_quality_check.py  
    _TEMPLATE.py
```

当插件依赖外部包时，应通过平台约定的运行环境预装依赖，或在插件元数据中声明依赖来源。插件代码不应在运行时自动执行 pip install。

5. 插件文件模板

```
# -*- coding: utf-8 -*-  
"""图像几何变换增强插件。"""  
  
from __future__ import annotations  
  
from pathlib import Path  
from typing import Any  
  
PARAMETERS: list[dict[str, Any]] = [  
    {  
        "name": "rotation_degrees",  
        "type": "float",  
        "label": "旋转角度",  
        "default": 10.0,
```

```

    "min": -180.0,
    "max": 180.0,
    "description": "图像旋转角度，单位为度。",
    "required": False,
},
{
    "name": "mode",
    "type": "select",
    "label": "增强模式",
    "default": "keep_size",
    "options": ["keep_size", "expand_canvas"],
    "description": "keep_size 表示保持原尺寸，expand_canvas 表示扩展画布。",
    "required": True,
},
]

```

```

def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    """执行算法插件。

```

Args:

payload: 平台下发的任务请求，包含 parameters、input、output 等字段。
context: 平台运行上下文，提供进度、日志和取消检查。

Returns:

标准插件返回对象。

```

"""

```

```

parameters = payload.get("parameters", {}) or {}
input_info = payload.get("input", {}) or {}
output_info = payload.get("output", {}) or {}

```

```

samples = input_info.get("samples", []) or []
output_dir = Path(output_info.get("output_dir", ""))

```

```

if not samples:

```

```

    return {
        "ok": False,
        "error_code": "NO_INPUT_SAMPLES",
        "message": "未收到可处理的输入样本。",
    }

```

```

output_dir.mkdir(parents=True, exist_ok=True)
context.set_progress(0, "开始处理")

```

```

outputs: list[dict[str, Any]] = []
total = len(samples)

```

```

for index, sample in enumerate(samples, start=1):

```

```

    if context.is_cancel_requested():
        return {
            "ok": False,
            "error_code": "CANCELLED",
            "message": "任务已取消。",
        }

```

业务处理逻辑在此实现。

```

context.log("info", "处理样本", {"sample_id": sample.get("id")})
context.set_progress(index * 100 / total, f"已处理 {index}/{total}")

```

```

return {
    "ok": True,
    "outputs": outputs,
    "logs": [],
}

```

6. 参数声明规范

6.1 PARAMETERS 基本要求

PARAMETERS 必须是模块级变量，类型为 `list[dict[str, Any]]`。平台通过该变量生成参数配置 UI，并在任务执行前进行基础校验。

即使插件不需要参数，也必须显式声明：

```
PARAMETERS: list[dict[str, Any]] = []
```

6.2 字段定义

字段	类型	必填	说明
name	str	是	参数变量名。必须使用英文小写字母、数字和下划线，且以字母开头，例如 blur_threshold。
type	str	是	参数类型。仅支持 string、int、float、bool、select。
label	str	是	UI 展示名称，应使用简短中文，例如模糊阈值。
default	任意	是	默认值，类型必须与 type 匹配。
min	int/float	否	数值下限，仅对 int 和 float 有效。
max	int/float	否	数值上限，仅对 int 和 float 有效。
options	list	条件必填	枚举项，仅对 select 有效，且不能为空。
description	str	否	参数说明文本，用于解释业务含义、单位和取值影响。
required	bool	否	是否必填，默认 False。

6.3 类型约束

type	default 示例	约束说明
string	"normal"	字符串类型。
int	100	整数类型，不得使用浮点数表达整数。
float	0.5	浮点数类型，可配置 min 和 max。
bool	True	布尔类型。
select	"A"	枚举选择，必须提供非空 options，默认值必须在 options 内。

平台不支持 number、integer、json 等非标准类型名。整数必须使用 int，浮点数必须使用 float。

6.4 参数校验规则

插件开发者应确保参数声明满足以下条件：

- name 在同一插件内唯一。
- name 不得使用 Python 关键字。
- label 不得为空，不建议超过 20 个中文字符。
- description 应说明单位、范围、默认行为和边界影响。
- 当声明 min 和 max 时，必须满足 min <= default <= max。
- 当 type 为 select 时，options 必须是非空列表，且所有选项应为字符串或可稳定序列化的简单类型。
- default 不得使用可变对象作为隐式共享状态；若需要列表或字典，应在 run() 中复制后使用。

7. 运行入口规范

每个插件必须提供以下函数签名：

```
def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    ...
```

run() 是插件唯一执行入口。平台负责构造 payload、注入 context、管理任务状态并收集返回结果。

run() 必须遵循以下约束：

- 不得修改输入样本原文件。
- 不得写入 output_dir 之外的业务产物。
- 不得吞掉异常后返回成功。
- 不得返回不可 JSON 序列化的对象，例如 Path、datetime、numpy.ndarray、文件句柄。
- 长耗时循环必须周期性调用 context.is_cancel_requested()。
- 任务进度应从 0 到 100 单调递增。

8. payload 结构规范

平台传入的 payload 推荐结构如下：

```
payload = {
  "algorithm_key": "generation.image.geometric_transform",
  "parameters": {
    "rotation_degrees": 10.0,
    "scale": 1.0,
  },
  "input": {
    "dataset_id": 1,
    "dataset_path": "/data/datasets/abc",
    "samples": [
      {
        "id": 1,
        "sample_path": "/data/datasets/abc/img_001.jpg",
        "modality": "image",
        "labels_json": ["ship"],
      }
    ],
  },
  "output": {
    "output_dir": "/data/tasks/42/output",
  },
  "target_count": 100,
}
```

8.1 顶层字段

字段	类型	必填	说明
algorithm_key	str	是	算法唯一标识。
parameters	dict	是	参数字典，键名对应 PARAMETERS.name。
input	dict	是	输入数据描述。
output	dict	是	输出目录描述。
target_count	int	否	期望生成数量，常用于增强和生成类插件。

8.2 input 字段

字段	类型	必填	说明
dataset_id	int/str	否	数据集标识。

字段	类型	必填	说明
dataset_path	str	否	数据集根路径。
samples	list	是	样本列表。

样本对象建议包含以下字段：

字段	类型	必填	说明
id	int/str	是	样本唯一标识。
sample_path	str	是	样本文件路径。
modality	str	否	数据模态，例如 image、text、audio、video。
labels_json	list/dict	否	样本标签，采用可 JSON 序列化结构。
metadata	dict	否	样本扩展信息。

8.3 output 字段

字段	类型	必填	说明
output_dir	str	是	平台为本次任务创建的输出目录。插件应将所有产物写入该目录。

插件应使用 `pathlib.Path` 处理路径，并使用 `/` 作为跨平台相对路径分隔符。

9. context 接口规范

平台通过 `context` 向插件提供运行期能力。插件不得假设 `context` 的具体实现类型，只能调用规范中定义的方法。

方法	说明
<code>context.set_progress(percent, message)</code>	上报任务进度。percent 范围为 0 到 100，message 为当前阶段说明。
<code>context.log(level, message, payload=None)</code>	记录结构化日志。level 支持 info、warn、error。
<code>context.is_cancel_requested() -> bool</code>	检查任务是否收到取消请求。长循环中必须周期性调用。

日志要求：

- 日志消息应清晰描述当前动作或异常原因。
- `payload` 必须可 JSON 序列化。
- 不得在日志中输出密钥、令牌、个人敏感信息或大体积二进制内容。
- 同类重复日志应聚合或降采样，避免刷屏。

10. 返回值规范

`run()` 必须返回 `dict[str, Any]`，且整体可 JSON 序列化。

10.1 成功返回：清洗建议类

```
{
  "ok": True,
  "suggestions": [
    {
      "sample_id": 1,
      "issue_type": "blur",

```

```
      "suggested_action": "repair",
      "confidence": 0.85,
      "message": "图像模糊度为 45.2, 建议修复。",
      "details": {
        "blur_score": 45.2,
        "output_path": "/path/to/repared.jpg",
      },
    },
  ],
  "logs": [],
}
```

10.2 成功返回：生成或增强类

```
{
  "ok": True,
  "outputs": [
    {
      "source_sample_id": 1,
      "output_path": "/data/tasks/42/output/img_001_geo_0000.jpg",
      "relative_path": "img_001_geo_0000.jpg",
      "metadata": {
        "method": "geometric_transform",
        "parameters": {"rotation_degrees": 10.0},
      },
      "status": "created",
    }
  ],
  "logs": [],
}
```

10.3 失败返回

```
{
  "ok": False,
  "error_code": "NO_INPUT_SAMPLES",
  "message": "未收到可处理的输入样本。",
}
```

10.4 取消返回

```
{
  "ok": False,
  "error_code": "CANCELLED",
  "message": "任务已取消。",
}
```

10.5 返回字段要求

字段	类型	必填	说明
ok	bool	是	执行是否成功。
outputs	list	条件必填	生成或增强类插件成功时返回。
suggestions	list	条件必填	清洗建议类插件成功时返回。
logs	list	否	插件自带日志摘要。详细日志推荐通过 context.log() 上报。
error_code	str	失败必填	稳定错误码，使用英文大写和下划线。
message	str	失败必填	面向用户或运维人员的可读说明。

11. 错误码规范

错误码必须稳定、可搜索、可统计。格式为英文大写字母、数字和下划线，例如 INVALID_PARAMETER。

推荐错误码如下：

错误码	使用场景
INVALID_PARAMETER	参数缺失、类型错误或取值越界。
NO_INPUT_SAMPLES	输入样本为空。
INPUT_FILE_NOT_FOUND	输入文件不存在。
UNSUPPORTED_MODALITY	数据模态不受支持。
OUTPUT_WRITE_FAILED	输出文件写入失败。
DEPENDENCY_MISSING	运行依赖缺失。
ALGORITHM_FAILED	算法内部处理失败。
CANCELLED	任务被取消。

异常处理建议：

```
try:
    ...
except ValueError as exc:
    context.log("error", "参数校验失败", {"error": str(exc)})
    return {
        "ok": False,
        "error_code": "INVALID_PARAMETER",
        "message": str(exc),
    }
except Exception as exc:
    context.log("error", "算法执行失败", {"error": str(exc)})
    return {
        "ok": False,
        "error_code": "ALGORITHM_FAILED",
        "message": "算法执行失败，请查看任务日志。",
    }
```

12. 参数读取推荐方式

插件应显式转换参数类型，避免平台、UI 或 JSON 反序列化导致的类型差异。

```
def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    parameters = payload.get("parameters", {}) or {}

    threshold = float(parameters.get("threshold", 0.5))
    max_iterations = int(parameters.get("max_iterations", 100))
    mode = str(parameters.get("mode", "normal"))
    enable_logging = _as_bool(parameters.get("enable_logging", True))
    stop_words = list(parameters.get("stop_words") or [])

    ...

def _as_bool(value: Any) -> bool:
    if isinstance(value, bool):
        return value
    return str(value).strip().lower() in {"1", "true", "yes", "y", "是"}
```

13. 命名规范

对象	规范	示例
参数 name	英文小写、数字、下划线，以字母开头	blur_threshold、learning_rate
参数 label	简短中文名称	模糊阈值、学习率
算法 key	点分命名空间	generation.image.geometric_transform
算法 name	UI 展示中文名称	几何变换增强
输出文件	可读、稳定、避免覆盖	img_001_geo_0000.jpg
错误码	英文大写、下划线	NO_INPUT_SAMPLES

14. 文件 I/O 规范

插件必须遵循以下文件操作要求：

- 输入文件只读，不得修改、删除或覆盖。
- 输出文件必须写入 `payload["output"]["output_dir"]` 或其子目录。
- 返回结果中必须同时提供绝对路径 `output_path` 和相对路径 `relative_path`。
- 相对路径应相对于 `output_dir`，并使用 `/` 分隔。
- 输出文件名应避免与原始文件冲突，批量生成时应包含序号或稳定哈希。
- 大文件处理应采用流式读取或分批处理，避免一次性加载全部数据。
- 临时文件应写入输出目录下的 `_tmp` 子目录，并在成功结束前清理。

15. 可观测性与运行控制

为支持平台调度、运维排障和用户反馈，插件必须具备基本可观测性。

进度上报要求：

- 任务开始时上报 0。
- 主要阶段切换时上报进度和阶段说明。
- 批量处理时按样本数量或工作量比例上报。
- 正常结束前上报 100。
- 进度不得倒退。

取消处理要求：

- 批量循环中至少每处理一个样本检查一次取消请求。
- 单个样本处理耗时较长时，应在内部关键步骤之间检查取消请求。
- 收到取消请求后，应尽快停止新增处理，并返回 `CANCELLED`。

16. 安全与隔离要求

插件不得执行以下行为：

- 读取与任务无关的系统敏感目录或用户私有目录。
- 将输入数据、参数或日志发送到未经授权的外部网络。
- 在运行时安装依赖、修改系统环境变量或启动长期后台进程。
- 执行来自参数、标签、文件名或外部输入的动态代码。
- 在日志、返回值或输出文件中泄露密钥、令牌、账户信息或个人敏感信息。

如插件确需访问网络、模型权重或外部服务，必须在交付说明中明确依赖、用途、地址、鉴权方式和失败降级策略，并由平台侧进行准入审批。

17. 性能与可靠性要求

插件应满足以下工程要求：

- 单个样本失败不应默认导致整个批次失败；除非该失败影响全局正确性。
- 对可恢复错误应记录日志并继续处理，对不可恢复错误应返回失败。
- 随机过程必须支持固定随机种子，确保可复现。
- 批量增强类插件应避免无限生成，生成数量必须受 `target_count` 或参数上限控制。
- CPU、内存、GPU、磁盘占用应与输入规模匹配，不得无界增长。
- 输出结果应具有确定的数据结构，便于平台消费和统计。

18. 测试要求

插件交付前至少应完成以下测试：

- 参数声明检查：PARAMETERS 字段完整、类型正确、默认值合法。
- 空输入测试：输入样本为空时返回 `NO_INPUT_SAMPLES`。
- 正常样例测试：使用最小样例数据可成功运行。
- 取消测试：模拟取消请求时返回 `CANCELLED`。
- 异常测试：输入文件缺失、参数越界等场景返回稳定错误码。
- 输出检查：所有产物位于 `output_dir` 内，路径可访问，返回值可 JSON 序列化。
- 兼容性测试：在平台目标 Python 版本和目标操作系统上运行通过。

推荐提供最小测试脚本或测试用例，便于平台集成验收。

19. 交付要求

插件交付物应包含：

- 插件 Python 文件。
- 示例参数配置。
- 最小输入样例或样例说明。
- 依赖清单及版本范围。
- 算法说明，包括用途、适用数据类型、关键参数、输出含义和限制。
- 测试记录，包括测试环境、测试数据、执行结果和已知问题。

交付前检查清单：

检查项	要求
接口	已声明 PARAMETERS，已实现 <code>run(payload, context)</code> 。
参数	字段完整，默认值合法，类型仅使用规范类型。
路径	不依赖当前工作目录，不写入输出目录之外。
返回值	成功和失败均符合标准结构，可 JSON 序列化。
日志	关键阶段有日志，不泄露敏感信息。
取消	长耗时流程支持取消。
测试	最小样例、异常场景、取消场景均通过。
文档	已说明用途、参数、输出、依赖和限制。

20. 兼容性与版本管理

插件升级应遵循兼容性原则：

- 新增可选参数属于兼容变更。
- 修改默认值可能影响结果，应在版本说明中明确。
- 删除参数、重命名参数、改变返回结构属于不兼容变更。
- 改变算法 Key 属于新插件，不应覆盖旧插件。
- 修复缺陷时应保留原有返回字段，避免破坏平台消费逻辑。

建议插件维护语义化版本，例如 1.2.0：

- 主版本号：不兼容变更。
- 次版本号：兼容性功能新增。
- 修订号：缺陷修复、性能优化或文档更新。

21. 附录：最小合规插件

```
# -*- coding: utf-8 -*-
"""最小合规 ISG 插件示例。"""

from __future__ import annotations
from typing import Any

PARAMETERS: list[dict[str, Any]] = []

def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
    samples = ((payload.get("input") or {}).get("samples")) or []

    if not samples:
        return {
            "ok": False,
            "error_code": "NO_INPUT_SAMPLES",
            "message": "未收到可处理的输入样本。",
        }

    context.set_progress(0, "开始处理")

    for index, sample in enumerate(samples, start=1):
        if context.is_cancel_requested():
            return {
                "ok": False,
                "error_code": "CANCELLED",
                "message": "任务已取消。",
            }
        context.log("info", "样本处理完成", {"sample_id": sample.get("id")})
        context.set_progress(index * 100 / len(samples), f"已处理 {index}/{len(samples)}")

    return {
        "ok": True,
        "outputs": [],
        "logs": [],
    }
```